



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 – SPECIAL TOPIC IN DATA ENGINEERING

SECTION 1

Tutorial 3: AI Algorithm Report

LECTURER: DR. ARYATI BINTI BAKRI

GROUP 3

STUDENT NAME	MATRIC NO
ELIJAH SHE YU SHENG	A23CS0073
CHERYL CHEONG KAH VOON	A23CS0060
CHUA JIA LIN	A23CS0069

TABLE OF CONTENTS

1.0	What is Image Classification?	3
2.0	What is CNN?	3
3.0	Evaluation of CNN	4
4.0	Steps Hand on in Python	5
5.0	What is the weakest in the current CNN model?	8
6.0	How to enhance the CNN model?	8
7.0	Comparison of CNN and new CNN model	11
8.0	Results and Discussion	14
9.0	Reflection	14

1.0 What is Image Classification?

Image Classification is one of the most important tasks in computer vision and artificial intelligence (AI). It refers to the process of assigning labels or categories to the image based on their visual content. The goal is to enable computers to automatically identify and identify objects, animals, people or scenes in digital images. For example, images containing cats can be classified as “cats”, while images containing cars can be classified as “cars”.

With the rapid growth of digital media and image data, image classification has been widely used in many real-world applications. These applications include facial recognition, biometric authentication, medical image analysis, self-driving cars, robots and security monitoring systems. Traditional image classification methods require manual feature extraction. Before classification, experts need to identify important image features. However, modern deep learning technology can automatically learn these characteristics and better performance.

The image classification process usually begins with collecting image datasets and performing data preprocessing to improve image quality and consistency. Then, the process image is used to train the classification model. During the training, the model learns important patterns and features from the image. After completing the training process, the model can classify new images that it has never seen before. Among the various deep learning methods available today, convolutional neural networks (CNN) have become the most popular and effective method of image classification because they can automatically learn complex image features.

2.0 What is CNN?

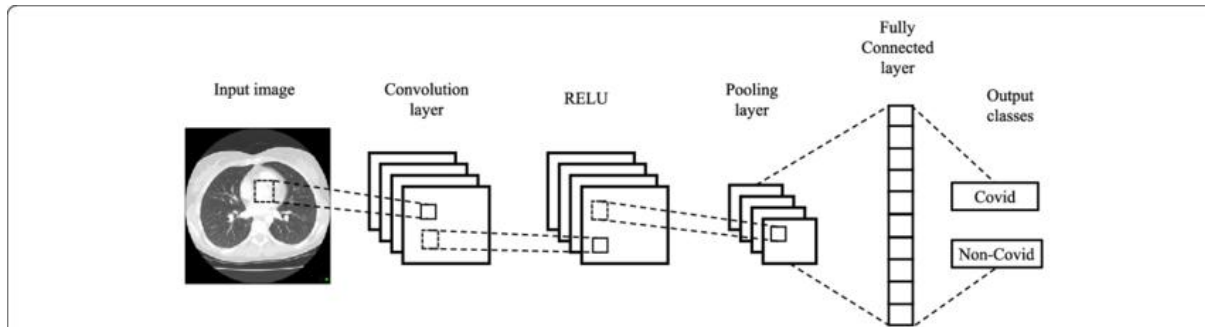
Convolutional neural network (CNN) is a deep learning model specially designed to process image data. CNN automatically learns important visual features such as edges, texture, shapes and object structure by applying mathematical operations called convolution. These learning functions enable the model to distinguish among different image categories and classify image accurately.

One of the main advantages of CNN is that it eliminates the need for manual feature extraction. CNN does not rely on human-designed functions but automatically learns the most relevant information from the training images. This ability makes CNN one of the most successful architectures in computer vision and has led to significant progress in applications such as image recognition, object detection, facial recognition, medical diagnosis and autonomous driving systems.

A typical CNN architecture consists of several layers which work together to classify images. The input layer receives image data and passes it to the convolution layer. The convolutional layer applied filters to extract important features from image and generate feature maps. After that, the rectifier linear unit (ReLU) activation function introduces nonlinearity into the

network so that it can learn more complex patterns. The pool layer reduces the size of the feature map while retaining important information and reducing the computing cost.

Then, use the flat layer to convert the extracted features into one-dimensional vectors. Finally, the fully connected layer performs classification tasks and generates predictive output classes.



CNN provides several advantages for image classification tasks. It automatically extracts image features, reduces the number of parameters through weight sharing, improves classification accuracy and run efficiently in large image datasets. These advantages make CNN one of the most widely used deep learning technologies in modern computer vision applications.

3.0 Evaluation of CNN

The performance of the CNN model was evaluated using several metrics including **accuracy**, **loss**, **precision**, **recall**, **F1-score**, and **confusion matrix**. Each metric provides different insights into the model's accuracy and reliability. The model's accuracy and reliability are very important for the performance of image classification tasks.

Accuracy and loss were used as the primary indicators of the model's overall performance. **Accuracy** measures the proportion of correctly classified samples out of the total datasets, showing the effectiveness of the model. An increasing accuracy during training indicates that the model is learning patterns in the training data, while a large gap between training accuracy and validation accuracy shows overfitting. However, accuracy does not reflect the misclassification patterns. On the other hand, **loss** measures the difference between predicted probabilities and actual classes. A decreasing loss during training indicates that the model is learning effectively, while a large gap between training and validation loss means overfitting is happening.

Precision, recall, and F1-score provide a deeper evaluation of the model's performance across different classes. **Precision** measures how many of the predicted positive classes were actually correct, while **recall** measures how many actual positive samples are correctly identified by the model. The **F1-score** combines both precision and recall metrics, offering a balanced evaluation, especially when class distributions are uneven. These metrics are useful in identifying whether the model makes more false positives or false negatives.

The **confusion matrix** provides the breakdown of predictions according to each class, showing correct and incorrect classifications for each category. The confusion matrix is useful for identifying which classes are frequently misclassified and understanding the patterns of misclassifications. By analyzing the confusion matrix, precision, and recall together, the model's weaknesses and classes that require improvement can be easily identified.

4.0 Steps Hand on in Python

This section demonstrates the implementation of an image classification model using Convolutional Neural Networks (CNNs) in Python using TensorFlow and Keras frameworks.

Import Required Libraries

The first step is importing the required libraries for model development, data processing and visualization. TensorFlow and Keras are used to build and train the CNN model, while Matplotlib is used to visualize the training results.

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

Load the CIFAR-10 Datasets

The CIFAR-10 datasets is loaded from the Keras dataset library. It contains 60000 color images categorized into 10 classes including airplane, automobiles, birds, cats, deer, dogs, frogs, horses, ships and trucks.

```
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
```

Data Preparation

The labels arrays are reshaped into one-dimensional vectors to simplify data handling. Several sample images are also displayed to verify that the dataset has been loaded correctly

```
y_train=y_train.reshape(-1,)
y_train[:5]
```

Show hidden output

```
y_test=y_test.reshape(-1,)
```

Data Normalization

The pixels values are normalized by dividing each value by 255.0. This scales the image data to a range between 0 and 1 which helps to improve the training performance and convergence speed.

```
x_train=x_train/255.0
x_test=x_test/255.0
```

Artificial Neural Network (ANN)

Before implementing CNN, basic Artificial Neural Network (ANN) model is constructed as a baseline for comparison. The ANN consists of a flatten layer and two dense hidden layers. The ANN model is used to generate predictions and evaluate classification performance through a classification report and confusion matrix.

```
ann = models.Sequential([
    layers.Flatten(input_shape=(32,32,3)),
    layers.Dense(3000, activation='relu'),
    layers.Dense(1000, activation='relu'),
    layers.Dense(10, activation='softmax')
])

ann.compile(optimizer='SGD',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])

ann.fit(x_train, y_train, epochs=5)
```

Convolutional Neural Network (CNN)

A CNN model is developed to improve image classification accuracy. The architecture consists of convolutional layers, pooling layers, flatten layers and dense layers.

```

cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3,3), activation='relu', input
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

```

Compile and Train the CNN model

The CNN model is compiled using the Adam optimizer and Sparse Categorical Crossentropy loss function. The model is trained for ten epochs using the training dataset.

```

cnn.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])

```

```

cnn.fit(X_train, y_train, epochs=10)

```

Evaluate the CNN model

The trained model is evaluated using the testing dataset to determine its classification accuracy.

```

cnn.evaluate(X_test, y_test)

```

Image Prediction

The CNN model predicts image classes from the testing dataset. The predicted label area compared with the actual labels to evaluate the model's performance.

```

y_pred = cnn.predict(X_test)
y_pred[:5]

```

[Show hidden output](#)

```

y_classes = [np.argmax(element) for element in y_pred]
y_classes[:5]

```

5.0 What is the weakest in the current CNN model?

The current CNN model is functional and can perform image classification on the CIFAR-10 dataset. However, it still has several weaknesses:

Weakness	Descriptions
Limited depth	The model only contains two convolution blocks, so it may not be able to learn deeper and more complex image features such as shapes and fine details.
No data augmentation	The model only learns from the original training images. This may reduce its ability to generalize well to new or unseen images.
No dropout layer	The model may overfit the training data, especially when it learns patterns that are too specific to the training set.
No batch normalization	The training process may be less stable and slower, which can affect model performance.
Limited evaluation using accuracy only	Accuracy alone does not show which image classes are commonly misclassified.

Therefore, an enhanced "new CNN" model is proposed to improve the performance of the current CNN model by using data augmentation, dropout, batch normalization, and more detailed evaluation metrics.

6.0 How to enhance the CNN model?

The enhanced new CNN improves the current CNN use:

Enhancement	Purpose
Data Augmentation	Creates variations of training images to improve generalization.
More Convolution Layers	Learns deeper and more complex visual features.
Batch Normalization	Stabilizes and speeds up training.
Dropout	Reduces overfitting.
Early Stopping	Stop training when validation performance stops improving.

Layer (type)	Output Shape	Param #
data_augmentation (Sequential)	(None, 32, 32, 3)	0
conv2d_2 (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
conv2d_3 (Conv2D)	(None, 32, 32, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_4 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256
conv2d_5 (Conv2D)	(None, 16, 16, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 16, 16, 64)	256
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_6 (Conv2D)	(None, 8, 8, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 8, 8, 128)	512
max_pooling2d_4 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
flatten_2 (Flatten)	(None, 2048)	0
dense_5 (Dense)	(None, 128)	262,272
batch_normalization_5 (BatchNormalization)	(None, 128)	512
dropout_3 (Dropout)	(None, 128)	0
dense_6 (Dense)	(None, 10)	1,290

Total params: 404,778 (1.54 MB)
Trainable params: 403,882 (1.54 MB)
Non-trainable params: 896 (3.50 KB)

```
early_stop = callbacks.EarlyStopping(
    monitor='val_loss',
    patience=4,
    restore_best_weights=True
)

new_cnn_history = new_CNN.fit(
    X_train, y_train,
    epochs=20,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)
```

```
new_cnn_loss, new_cnn_accuracy = new_CNN.evaluate(X_test, y_test, verbose=0)
print(f"new_CNN Test Accuracy: {new_cnn_accuracy:.4f}")
print(f"new_CNN Test Loss: {new_cnn_loss:.4f}")
```

```
new_CNN Test Accuracy: 0.7823
new_CNN Test Loss: 0.6323
```

```
new_cnn_pred = evaluate_model(new_CNN, X_test, y_test, class_names, title="new_CNN")
```

```
... Classification Report for new_CNN:
```

	precision	recall	f1-score	support
airplane	0.79	0.84	0.81	1000
automobile	0.91	0.91	0.91	1000
bird	0.75	0.63	0.68	1000
cat	0.64	0.65	0.65	1000
deer	0.74	0.71	0.73	1000
dog	0.85	0.58	0.69	1000
frog	0.63	0.93	0.75	1000
horse	0.84	0.83	0.84	1000
ship	0.92	0.86	0.89	1000
truck	0.84	0.88	0.86	1000
accuracy			0.78	10000
macro avg	0.79	0.78	0.78	10000
weighted avg	0.79	0.78	0.78	10000

Figure 6.0.1 Model Summary, Figure 6.0.2 Early Stopping Coding & Figure 6.0.3 Classification Report

Figure 6.0.1 on the left shows that :

1. The new CNN architecture is deeper than the original CNN model.
2. It includes more convolution layers, batch normalization layers, dropout layers, and dense layers.
3. This allows the model to learn more complex image features and helps improve its overall performance.

The code on the top right shows the use of Early Stopping.

1. It monitors the `val_loss`, which means validation loss.
2. If the validation loss does not improve after several epochs, the training process will stop automatically.
3. The setting `restore_best_weights = True` ensures that the model restores the best weights from the epoch with the best validation performance.

The result on the bottom right shows the evaluation of the new CNN model.

1. It includes test accuracy and test loss.
2. It shows precision, recall, and F1-score for each image class.
3. It shows how well the model performs for each category, not only the overall accuracy.

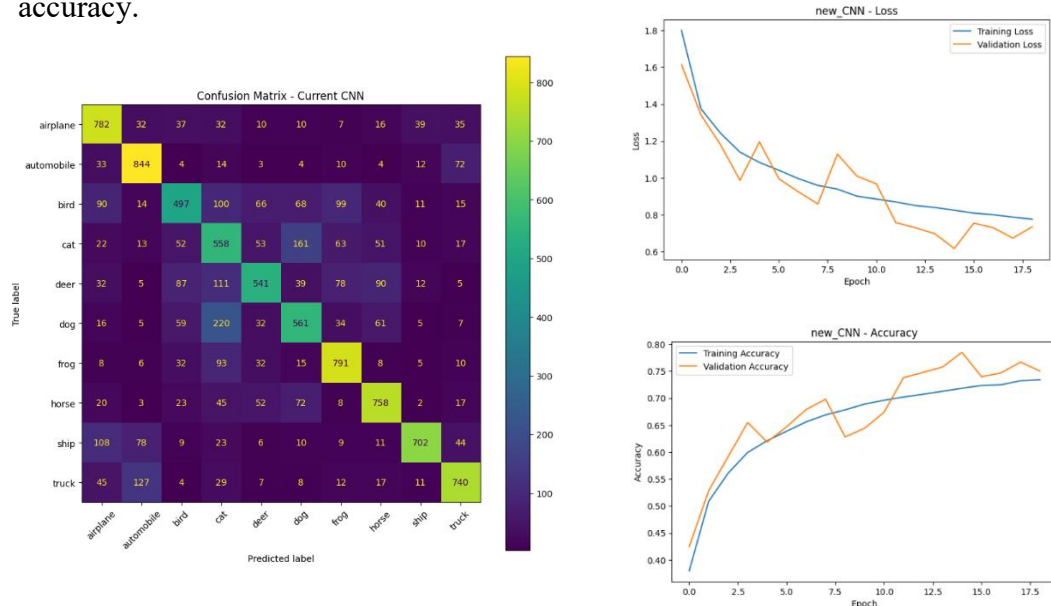


Figure 6.0.4 Predicted Label, Figure 6.0.5 new CNN loss Graph & Figure 6.0.6 new CNN Accuracy Graph

- Confusion matrix shows the prediction result for each CIFAR-10 class.
- Diagonal values represent correct predictions.
- The model performs well in classes such as automobile, frog, horse, ship, and truck.
- Some mistakes happen between visually similar classes, such as cat vs dog.
- The loss graph shows that training loss and validation loss generally decrease, meaning the new CNN is learning.
- The accuracy graph shows that training accuracy and validation accuracy increase over time.

Overall, the enhanced new CNN shows better learning performance and more stable training.

7.0 Comparison of CNN and new CNN model

In this section, a comparison between the CNN and the new CNN models was conducted on several evaluation metrics including accuracy, loss, precision, recall, F1-score, and confusion matrix. Figure 7.1 and Figure 7.2 show the evaluation results of the original CNN model, while Figure 7.3 and Figure 7.4 show the evaluation results of the new CNN model. Table 7.1 shows the comparison table for the original CNN and the new CNN model.

Table 7.1 Comparison Table for Original CNN and New CNN Model

Evaluation Metrics	Original CNN model	New CNN model
Accuracy	67.74%	78.23%
Loss	0.9881	0.6323
Precision	ship (0.87) truck (0.77) automobile (0.75)	ship (0.92) truck (0.84) automobile (0.91)
Recall	automobile (0.84) frog (0.79) airplane (0.78)	automobile (0.91) frog (0.93) airplane (0.84)
F1-score	0.68	0.78
Confusion Matrix	automobile (844) frog (791) airplane (782)	automobile (912) frog (929) airplane (840)

Based on the comparison results, the **new CNN model** shows **better performance** compared to the original CNN model in all evaluation metrics. This indicates a great improvement in both predictive accuracy and classification ability.

In terms of **overall performance**, the **accuracy** improved from **67.74%** (original CNN model) to **78.23%** (new CNN model). At the same time, the **loss** decreased from **0.9881** to **0.6323**. This shows that the new_CNN model produces more accurate predictions with lower errors during testing.

For **class-specific performance**, the new CNN model achieved higher precision and recall across all classes. **Precision** improved significantly from **0.75** to **0.91** for automobile, **0.77** to

0.84 for truck, and **0.87** to **0.92** for ship, indicating fewer false positive predictions. Similarly, **recall** also improved in classes such as frog (**0.79** to **0.93**), automobile (**0.84** to **0.91**), and airplane (**0.78** to **0.84**), showing that the model was able to correctly identify more true instances of each class. As a result, the overall **F1-score** increased from **0.68** to **0.78**, indicating a better balance between precision and recall in the new CNN model.

The **confusion matrix** further confirms the improvements from the original CNN model to the new CNN model, as the number of correctly classified images increased across all classes. For example, correct classifications improved from **844** to **912** for automobile, **791** to **929** for frog, and **782** to **840** for airplane, indicating that the new CNN model reduced misclassifications and strengthened its class discrimination ability. Overall, the comparison clearly shows that the new CNN model is more accurate and effective in learning meaningful image features than the original CNN model.

Current CNN Test Accuracy: 0.6774				
Current CNN Test Loss: 0.9881				
Classification Report for Current CNN:				
	precision	recall	f1-score	support
airplane	0.68	0.78	0.73	1000
automobile	0.75	0.84	0.79	1000
bird	0.62	0.50	0.55	1000
cat	0.46	0.56	0.50	1000
deer	0.67	0.54	0.60	1000
dog	0.59	0.56	0.58	1000
frog	0.71	0.79	0.75	1000
horse	0.72	0.76	0.74	1000
ship	0.87	0.70	0.78	1000
truck	0.77	0.74	0.75	1000
accuracy			0.68	10000
macro avg	0.68	0.68	0.68	10000
weighted avg	0.68	0.68	0.68	10000

Figure 3.1 Original CNN model's Evaluation Result

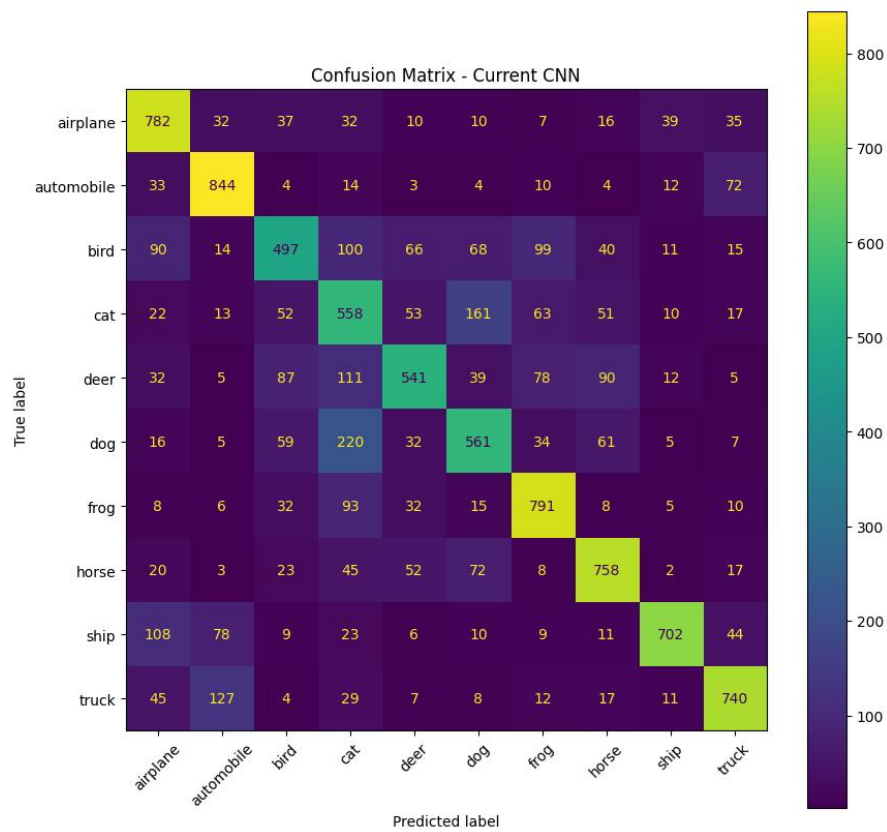


Figure 3.2 Original CNN Model's Confusion Matrix

new_CNN Test Accuracy: 0.7823

new_CNN Test Loss: 0.6323

Classification Report for new_CNN:

	precision	recall	f1-score	support
airplane	0.79	0.84	0.81	1000
automobile	0.91	0.91	0.91	1000
bird	0.75	0.63	0.68	1000
cat	0.64	0.65	0.65	1000
deer	0.74	0.71	0.73	1000
dog	0.85	0.58	0.69	1000
frog	0.63	0.93	0.75	1000
horse	0.84	0.83	0.84	1000
ship	0.92	0.86	0.89	1000
truck	0.84	0.88	0.86	1000
accuracy			0.78	10000
macro avg	0.79	0.78	0.78	10000
weighted avg	0.79	0.78	0.78	10000

Figure 3.3 New CNN model's Evaluation Result

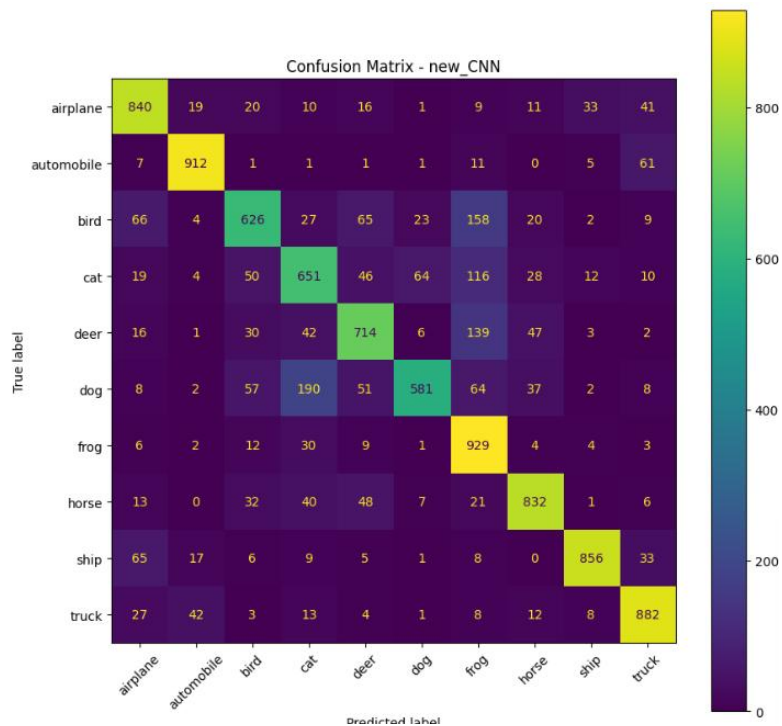


Figure 3.4 New CNN Model's Confusion Matrix

8.0 Results and Discussion

From the experiment, the enhanced new CNN model is expected to perform better than CNN model. The enhanced new CNN model can improve generalization because it uses data augmentation, dropout, batch normalization, and a deeper architecture. These enhancements help reduce overfitting and allow the model to learn more meaningful visual patterns. However, CIFAR-10 is still challenging because images are small and some classes look visually similar. For example, cat vs dog, deer vs horse, and automobile vs truck can be confusing for the model.

9.0 Reflection

Chua Jia Lin

Through this tutorial, a better understanding of using the convolutional neural network (CNN) for image classification was gained. At the beginning, it was difficult to understand how the convolution, flatten, dense, and max pooling layers work together to process the image data. However, through exploration on the Internet, the functions of these CNN layers became clearer.

The main challenge faced during this tutorial was improving the model's training efficiency. Training a deeper CNN model requires careful design of layer combinations and adjustment of parameters. To develop a deeper and more reliable CNN model, several architectures with different configurations and parameters were trained and evaluated using multiple evaluation metrics such as accuracy and loss. This ensured that the model achieved better performance in both overall and class-specific aspects. This process further enhanced practical experience in hands-on CNN model training.

For future improvement, more training data and better parameter tuning could be used to improve the model accuracy and reduce overfitting.

Cheryl Cheong Kah Voon

This tutorial provides practical contact with image classification using deep learning technology. The implementation process covers important stages, such as data set preparation, data rationalization, model development, training and evaluation using CIFAR-10 data sets. Through these activities, we have a better understanding of how machine learning models process image data and learn classification patterns. The experiment also demonstrates the importance of appropriate data preprocessing in improving model performance and training efficiency.

In addition, the comparison between artificial neural networks (ANN) and convolutional neural networks (CNN) highlights the advantages of CNN in image classification tasks. CNN automatically extracts important features from images through convolution and pool operations, thus achieving better performance. Overall, the tutorial successfully demonstrates the practical application of deep learning in computer vision and provides valuable knowledge about using Python and TensorFlow to develop and evaluate image classification models.

Elijah She Yu Sheng

This tutorial provided a better understanding of how Convolutional Neural Networks (CNN) are used for image classification. The implementation showed how convolutional layers, pooling layers, flatten layers, and dense layers work together to extract image features and classify images. It also highlighted the importance of improving a model, as the original CNN had weaknesses such as limited depth, no data augmentation, no dropout, and no batch normalization.

After applying improvements such as data augmentation, deeper layers, batch normalization, dropout, and early stopping, the new CNN model achieved better performance, with accuracy increasing from 67.74% to 78.23% and loss decreasing from 0.9881 to 0.6323. Overall, this tutorial strengthened practical skills in CNN model evaluation, while showing the importance

of using accuracy, precision, recall, F1-score, and confusion matrix to better understand model performance.